

5. Expresiones regulares

5.1. Definición y uso

Las expresiones regulares, también conocidas como *regexp* o *regex*, son **secuencias de caracteres** que especifican un **patrón de búsqueda**. Se utilizan para **localizar** o **filtrar** cadenas de **caracteres** que satisfagan un conjunto de **condiciones**.

Es muy habitual utilizarlas para:

- Realizar **búsquedas** en un listado.
- **Validar** campos de formulario.
- **Eliminar** o **Ignorar** caracteres no válidos
- **Ignorar** mayúsculas, minúsculas o acentos en las búsquedas.
- **Limitar** el número mínimo o máximo de caracteres.

! IMPORTANTE

No es objetivo de este apartado realizar un estudio pormenorizado de las expresiones regulares, dado que daría para un libro completo sobre el tema.

EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Enlaces web*, puedes consultar *The Modern JavaScript Tutorial*, donde hay un capítulo muy detallado dedicado a este tema, y *Regular Expressions*, de MDN, donde también hay abundante información.

Las expresiones regulares en JavaScript se pueden definir de dos maneras:

- Notación **constructora**. `regexp = new RegExp("patrón", "banderas");`
- Notación **literal**. `regexp = /patrón/banderas;`

5.2. Patrones

El **patrón** de una expresión regular está definido por un conjunto de **caracteres** que pueden ser **simples** o **especiales**. Los caracteres simples representan su significado literal, mientras que los caracteres **especiales** tienen un **comportamiento asociado**. Por ejemplo, el carácter simple **9** representa al número 9, mientras que el carácter **.** representa a cualquier carácter menos el salto de línea.

A continuación, se muestra un listado de los **grupos de caracteres especiales** más utilizados:

- Clases de caracteres. Se utilizan para **distinguir** caracteres:
 - **.** – Cualquier carácter (menos saltos de línea).
 - **\d** – Cualquier dígito numérico.
 - **\D** – Cualquier carácter **menos** un dígito numérico.
 - **\w** – Cualquier carácter alfanumérico.
 - **\s** – Cualquier espacio en blanco, incluyendo números de línea.
- Cuantificadores. Se utilizan para indicar un **número de caracteres** que hay que seleccionar:
 - ***** – 0 o más veces el ítem anterior.
 - **+** – 1 o más veces el ítem anterior.
 - **?** – 0 veces o 1 vez el ítem anterior.
 - **{n}** – *n* veces el ítem anterior.

- Grupos y rangos. Se utilizan para **agrupar** caracteres:
 - `[]` – Lista o rango de caracteres entre los que elegir: `/[LT]ORO/` hará referencia a **LORO** y a **TORO**.
 - `()` – Para crear **grupos de captura**: se utilizan para encontrar y **almacenar** grupos de caracteres y para que los **cuantificadores afecten al grupo**, no al carácter `/hola{2}/` hace referencia a `"holaa"`, mientras que `/(hola){2}/` hace referencia a `"holahola"`.
- Aserciones. Se utilizan para hacer referencia a **límites**:
 - `^` – Hace referencia al **comienzo** de la cadena.
 - `$` – Hace referencia al **final** de la cadena.
 - `\b` – Límites de palabra.

Si se desea utilizar un carácter especial como carácter simple, se debe **escapar** utilizando el carácter `\` delante del carácter especial. Así, si queremos hacer referencia al carácter `*` como carácter simple, deberemos escribir `\*`.

5.3. Banderas

Además de los patrones, las expresiones regulares pueden definir **banderas** o *flags* que afectan a su funcionamiento. Dichas banderas se pueden combinar unas con otras. Son:

- **g** – Búsqueda **global**. Si no se utiliza, la expresión regular solo detecta la **primera ocurrencia**.
- **i** – No distinguir mayúsculas de minúsculas.
- **m** – Búsqueda **multilínea**.
- **s** – Permite que el carácter `.` haga referencia también a caracteres de salto de línea.
- **u** – Para buscar en código Unicode.
- **y** – Búsqueda **pegajosa** (*sticky*): para buscar a partir de la posición en la que se quedó la última búsqueda.

5.4. Funciones para utilizar con expresiones regulares

Las expresiones regulares son **objetos** cuyo prototipo es **RegExp.prototype**, por lo que tienen acceso a una serie de **métodos** y **propiedades** definidos en dicho prototipo. Algunas de las **propiedades** más interesantes son:

- **lastIndex**. Lugar donde ha finalizado la búsqueda anterior. Si se utiliza la bandera **g**, siempre es 0.
- **ignoreCase**. Indica si se ha utilizado la bandera **i**.
- **global**. Indica si se ha utilizado la bandera **g**.
- **multiline**. Indica si se ha utilizado la bandera **m**.

! IMPORTANTE

Las propiedades anteriores son de **solo lectura**.

Además, los **strings** de JavaScript también tienen acceso a una serie de métodos para trabajar con expresiones regulares. A continuación, se muestran algunos de los **métodos** más utilizados para ello, indicando si se ejecutan sobre **strings** (**str**) o expresiones regulares (**regexp**):

- **str.search(regexp)**. Busca sobre la cadena indicada según la expresión regular indicada como parámetro. Devuelve la **posición** de la **primera coincidencia**, o **-1**, si no se encuentra.
- **str.match(regexp)**. Devuelve las ocurrencias de la expresión regular en la cadena dada. **Siempre devuelve un array**: si se utiliza **g**, cada valor del array es una de las ocurrencias encontradas; si no se utiliza **g**, se devuelve igualmente un array, cuya primera posición es la ocurrencia encontrada.
- **str.replace(regexp, reemplazo)**. Reemplaza las coincidencias del patrón con **reemplazo**, que puede ser una cadena o una función.
- **regexp.test(str)**. Devuelve **true** o **false** en función de si la expresión regular encuentra una coincidencia o no.

5.5. Ejemplos

A continuación, se muestran algunos ejemplos básicos del uso de expresiones regulares:

```
var cadenal = "Hola mundo! Hola de nuevo! Probaremos diferentes cadenas,
como AAbCC o rrXbCC o i98AA34bCC";

/hola/.test(cadenal); // false
/hola/i.test(cadenal); // true (bandera 'i')

cadenal.replace(/Hola/, "Hello"); // 'Hello mundo! Hola de nuevo! ...'
cadenal.replace(/Hola/g, "Hello"); // 'Hello mundo! Hello de nuevo! ...'

cadenal.match(/bCC/); // Array cuya primera posición es 'bCC'
cadenal.match(/bCC/g); // [ 'bCC', 'bCC', 'bCC' ]

cadenal.match(/\w*!/); // Array cuya primera posición es 'mundo!'
cadenal.match(/\w*!/g); // [ 'mundo!', 'nuevo!' ]

cadenal.match(/\w*bCC/); // Array cuya primera posición es 'AAbCC'
cadenal.match(/\w*bCC/g); // [ 'AAbCC', 'rrXbCC', 'i98AA34bCC' ]
cadenal.replace(/\w*bCC/g, "c"); // 'Hola mundo! Hola de nuevo! Probaremos
diferentes cadenas, como c o c o c'
```

A continuación, se muestra un ejemplo de cómo obtener números de teléfono escritos en varios formatos:

```
// Filtrar números de teléfono
var tel1 = "612-443-355";
var tel2 = "612 44 33 55";
var tel3 = "(612) 44 33 55";
var regTel = /\d/g; // Se buscan caracteres numéricos

// 'match' devuelve un array con las coincidencias; 'join' combina los
elementos de un array
// con el delimitador indicado
tel1.match(regTel).join(""); // '612443355'
tel2.match(regTel).join(""); // '612443355'
tel3.match(regTel).join(""); // '612443355'

// Otra opción con 'replace'. '\D' hace referencia a dígitos no numéricos
// Se reemplazan los dígitos no numéricos con el carácter vacío
var regTel2 = /\D/g; // Se buscan caracteres numéricos
tel1.replace(regTel2, ""); // '612443355'
tel2.replace(regTel2, ""); // '612443355'
tel3.replace(regTel2, ""); // '612443355'
```



EJEMPLO

En la documentación complementaria de la unidad, situada en la sección *Material de trabajo / Recursos adicionales*, puedes acceder a *Regular expressions 101*, un sitio web para crear, probar y practicar las expresiones regulares.